

TDVI: Inteligencia Artificial

Transformers

Licenciatura en Tecnología Digital

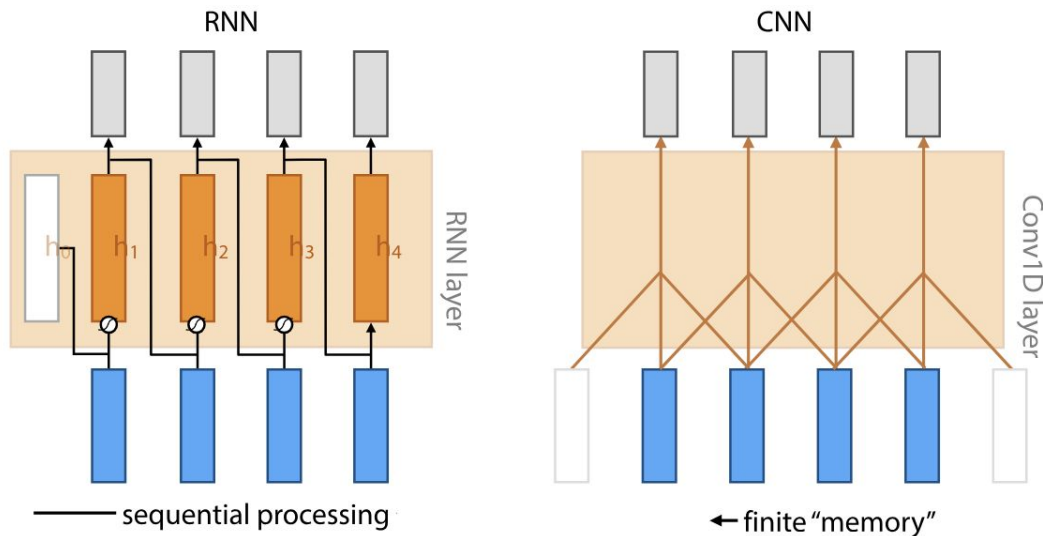


Agenda

1. **Attention**
2. Transformers
3. BERT
4. Vision Transformer (ViT)
5. Bibliografía

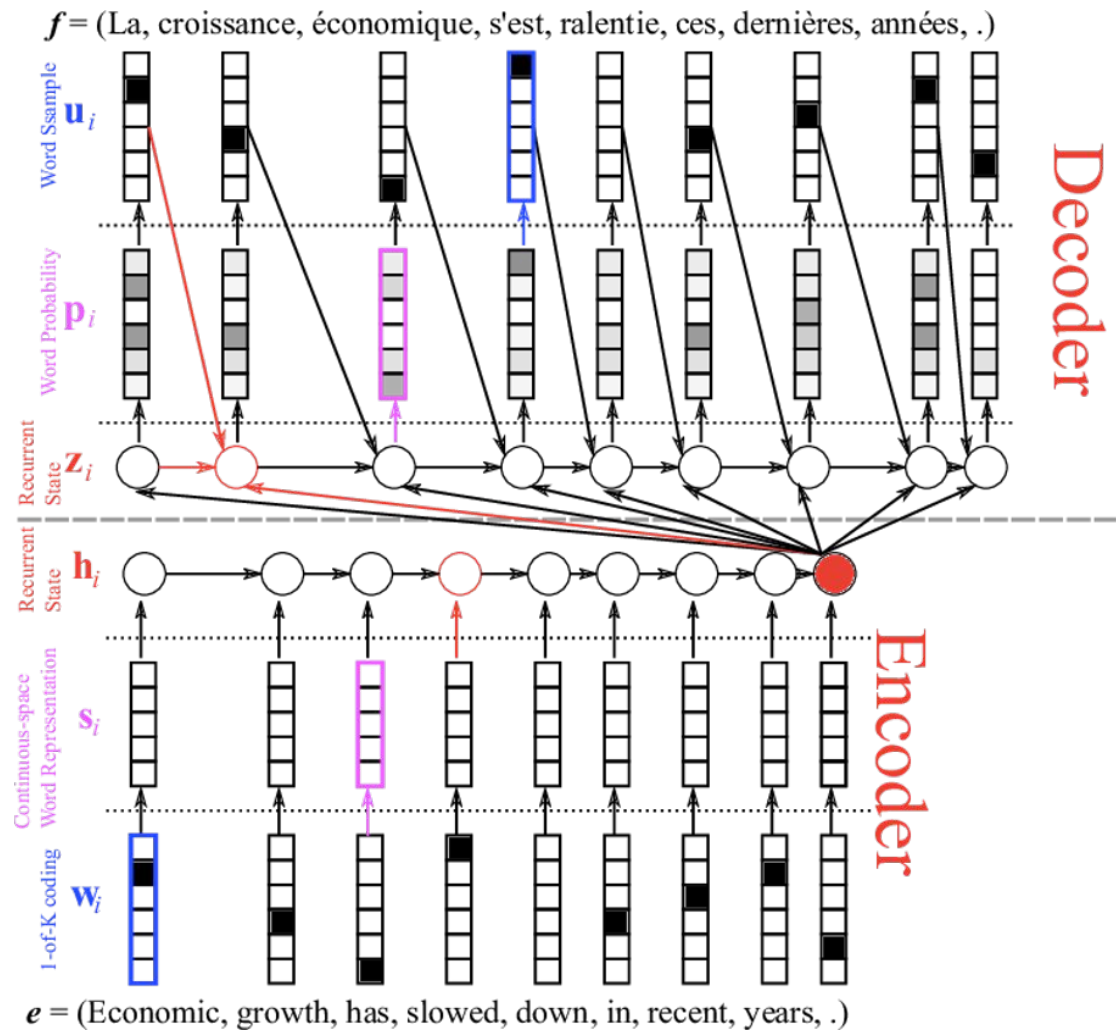
Secuencia a Secuencia

- Las RNNs pueden mirar hacia atrás infinitamente en la secuencia, pero son lentas debido al procesamiento secuencial.
- Las convoluciones pueden calcular en paralelo, pero tienen limitaciones en cuanto a retroceder en la secuencia.

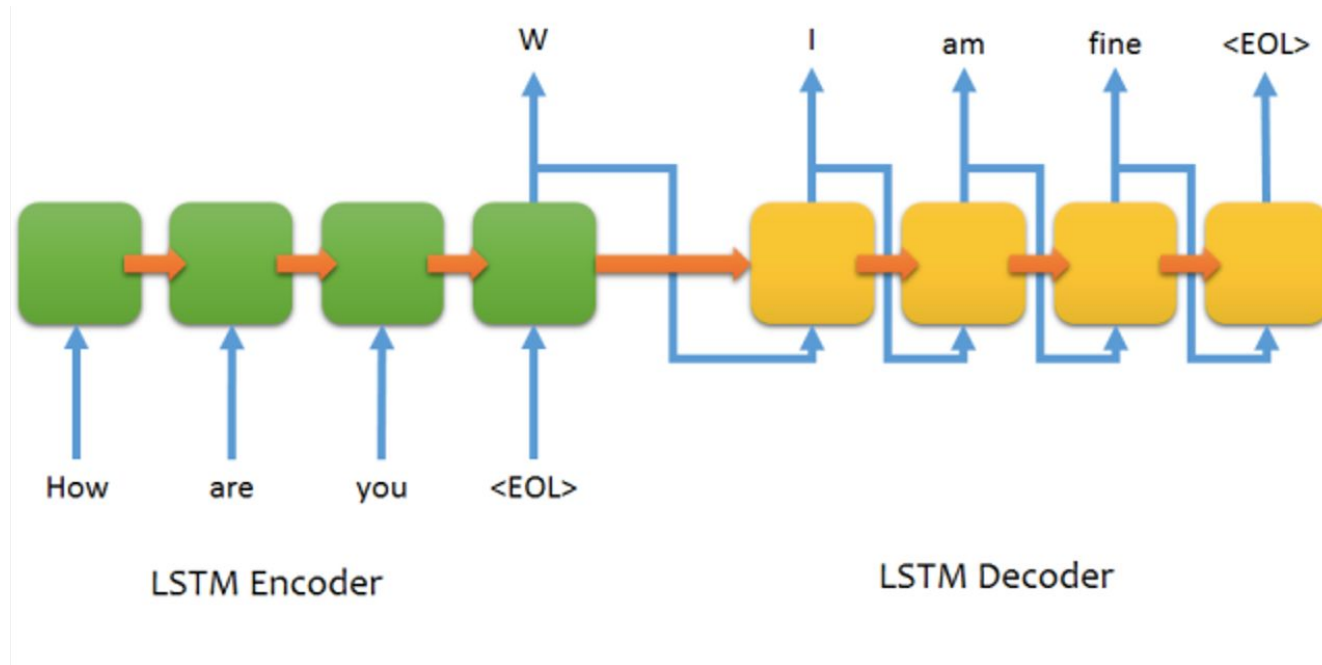


Encoder-Decoder machine translation

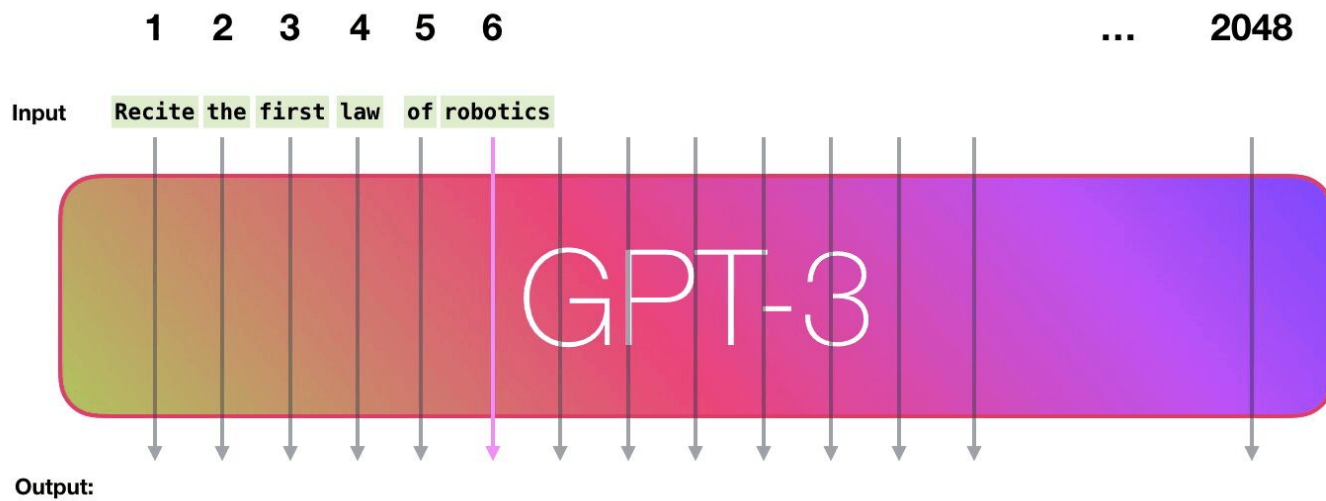
(LSTM)



Encoder-Decoder LSTM structure for chatting



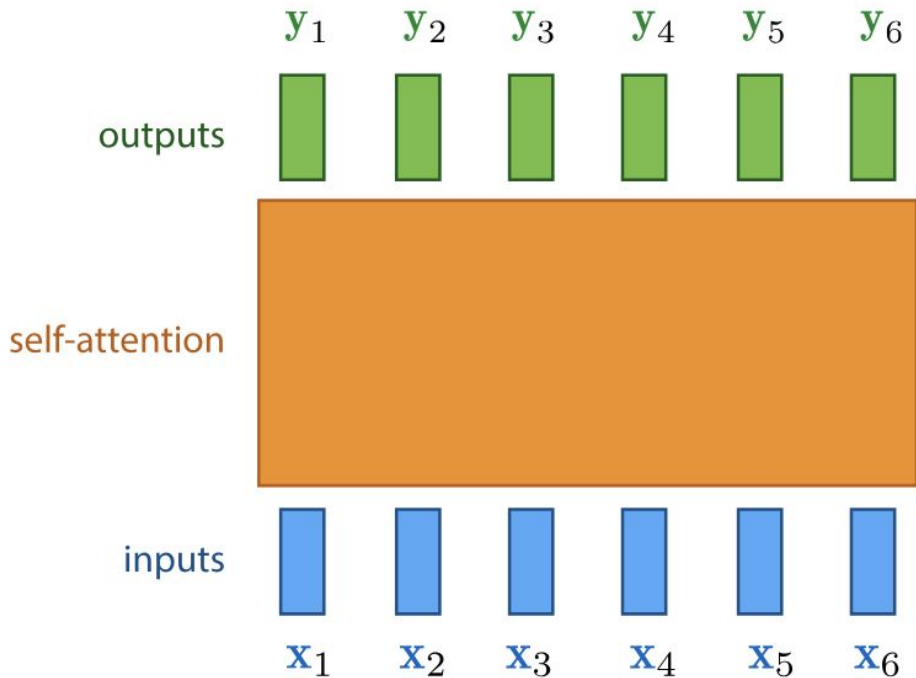
GPT



Self attention

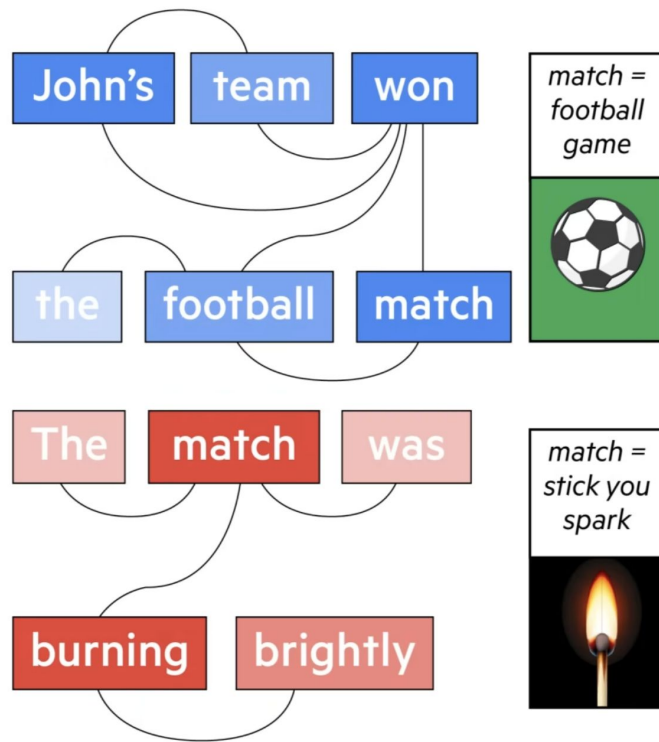
Necesitaremos:

- Un mecanismo que pueda tener en cuenta todo el contexto del codificador
- Que se actualice dinámicamente durante el proceso de decodificación
- Que pueda ser representado en un vector de atención de tamaño fijo.

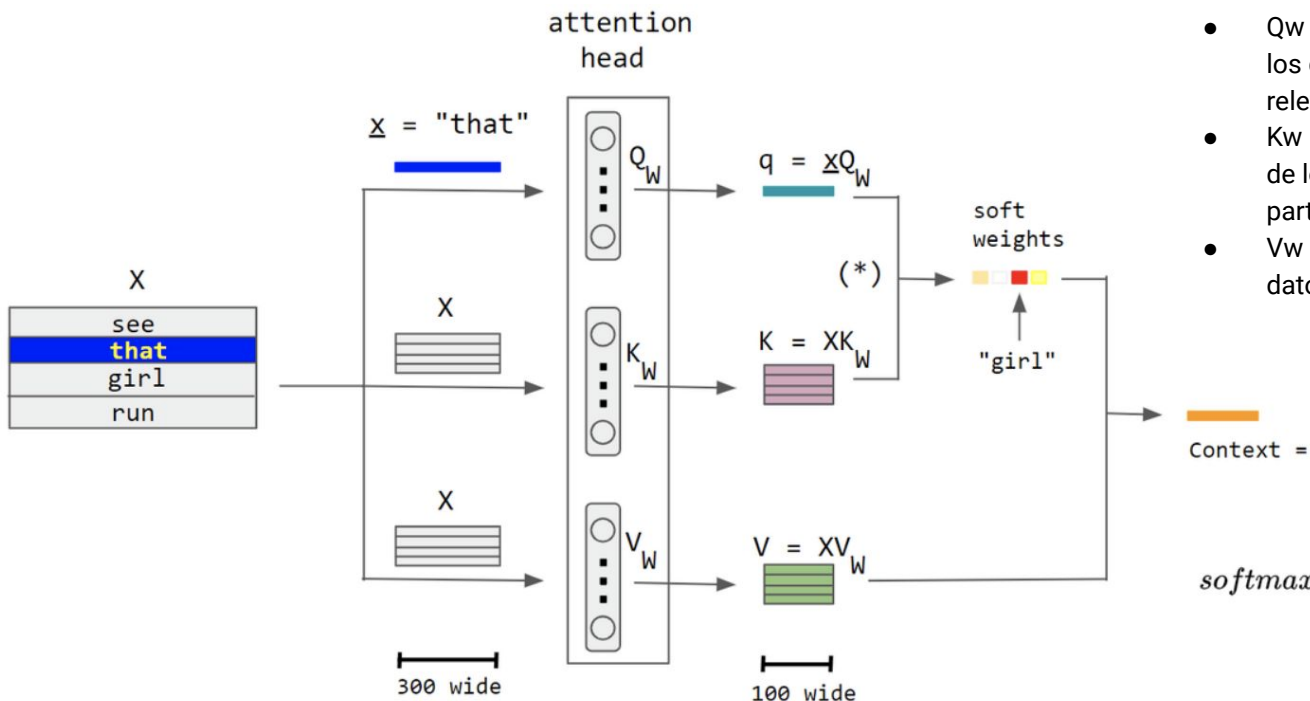


Attention

- La "atención" es un mecanismo que permite a la red centrarse en partes específicas de la entrada.
- Proporciona la capacidad de asignar niveles de importancia variables a diferentes partes de la información de entrada.
- Es particularmente útil en tareas que involucran secuencias de datos
- Facilita una mejor comprensión del contexto y la secuencialidad.



Attention



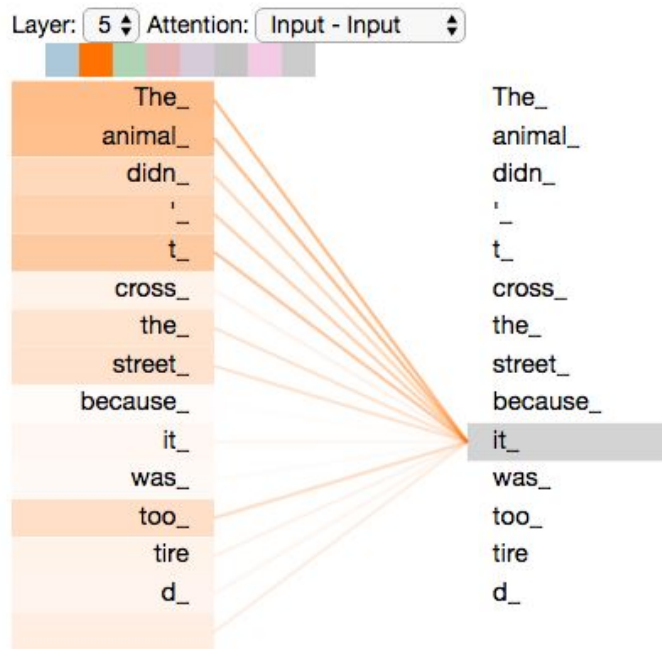
- Q_W (Pesos de Consulta): Capturan la estructura de los datos de entrada para determinar partes relevantes.
- K_W (Pesos de Clave): También capturan la estructura de los datos y establecen relaciones entre diferentes partes.
- V_W (Pesos de Valor): Expresan la estructura de los datos de manera más útil para la tarea específica.

$$\text{Context} = \text{softmax}\left(\frac{\overbrace{\underline{x}Q_w}^q * \overbrace{XK_w^T}^{K^T}}{\sqrt{100}}\right) * \overbrace{XV_w}^v$$

- X : Denota una matriz de tamaño 4×300 . (embeddings) de las cuatro palabras.
- $\underline{x}$: vector de embedding para la palabra "that". Este vector tiene un tamaño de 300.

Ejemplo

"The animal didn't cross the street because it was too tired"

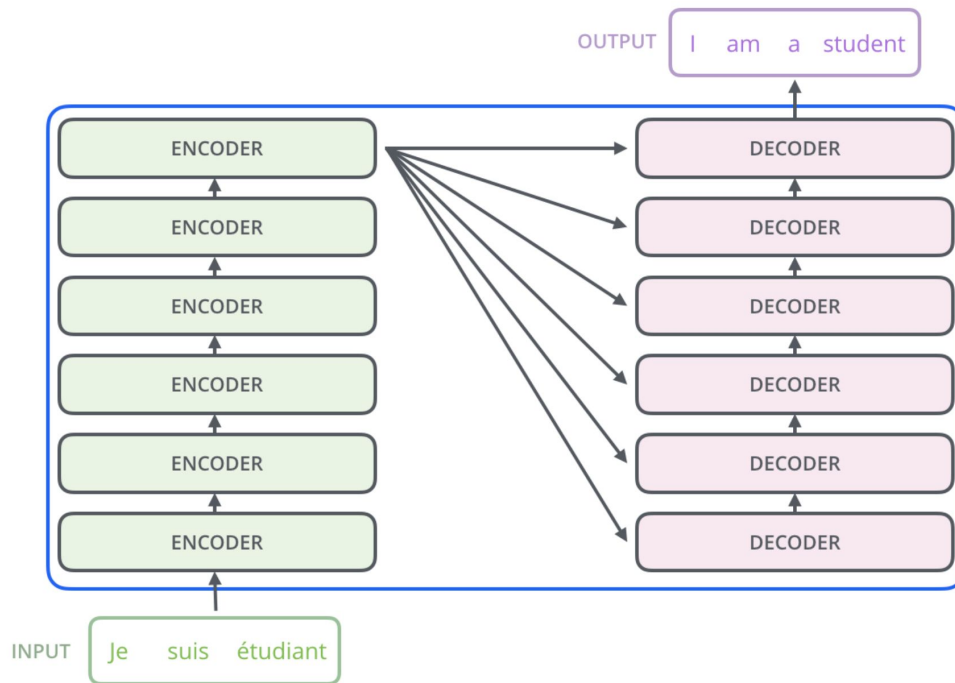


Agenda

1. Attention
2. **Transformers**
3. BERT
4. Vision Transformer (ViT)
5. Bibliografía

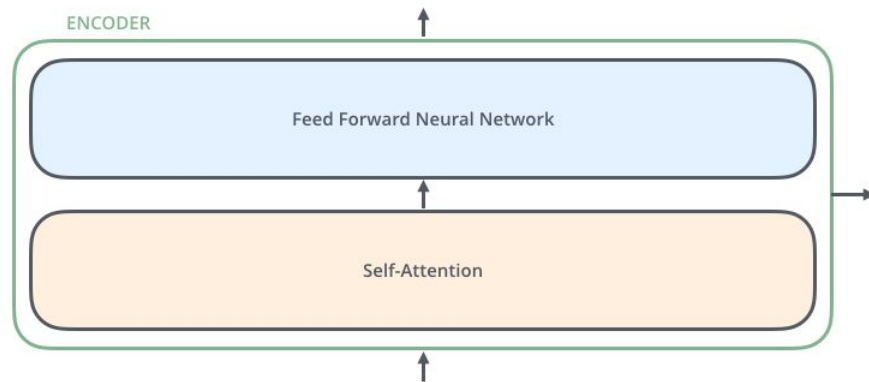
Transformers

- 6 Encoders apilados.
- 6 Decoders apilados.



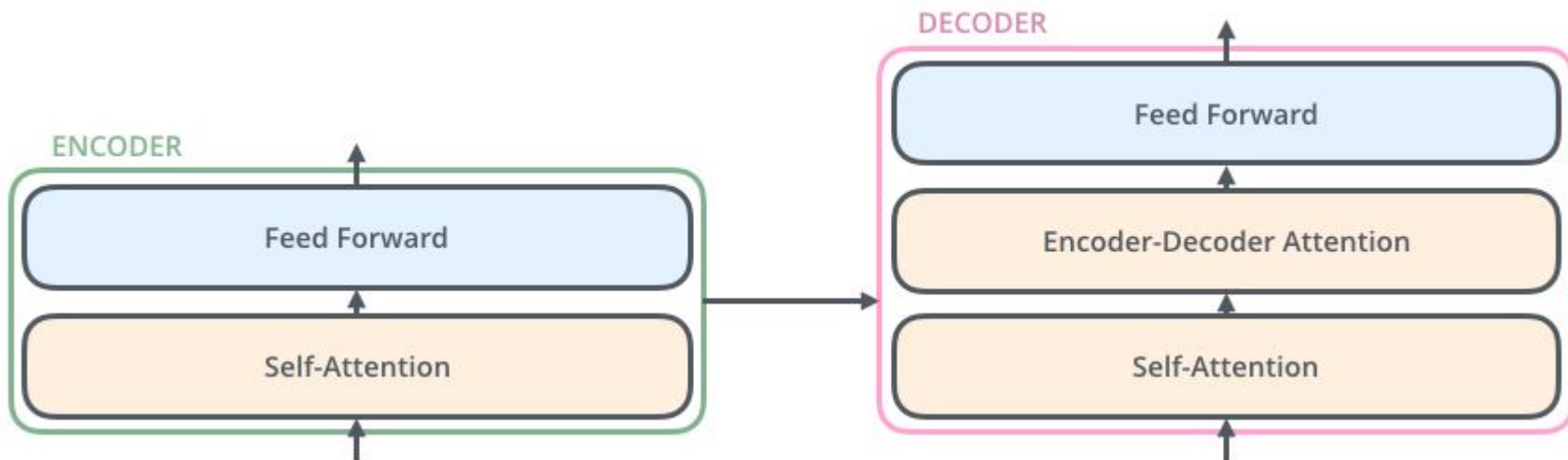
Encoder

- Self-Attention en el Codificador:
- Ayuda al codificador a examinar otras palabras en la oración de entrada mientras codifica una palabra específica.
- Feed-Forward NN en el Codificador:
- se aplica de manera independiente a cada posición.



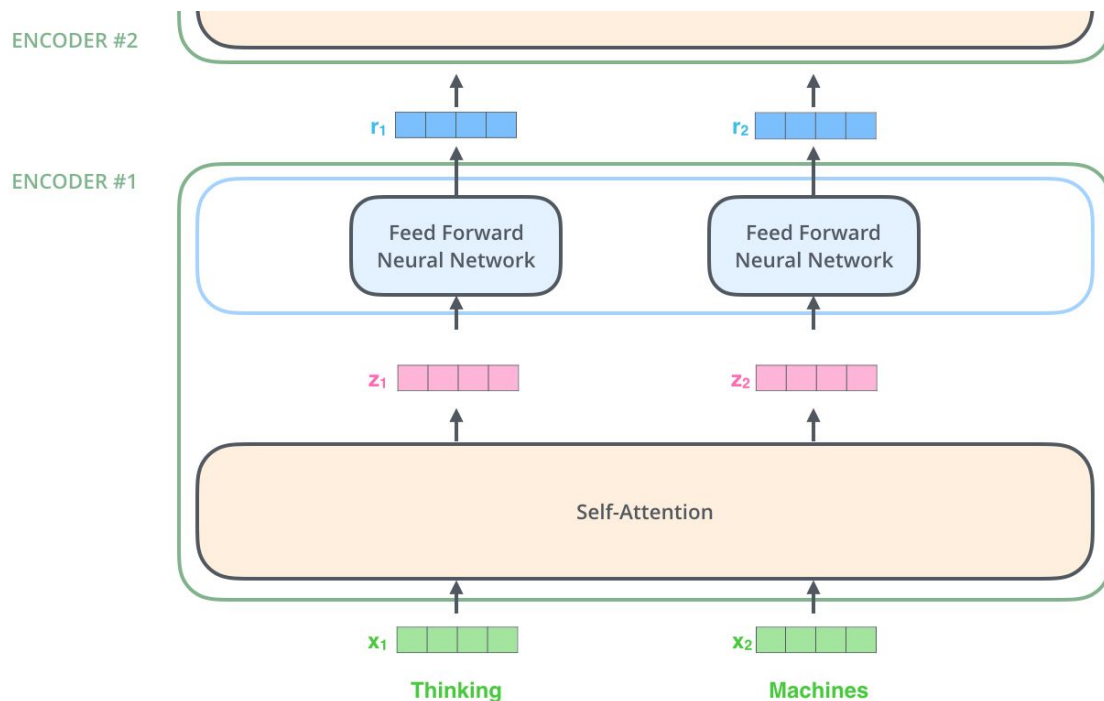
Decoder

- El decodificador incluye ambas capas del encoder
- Entre estas capas, hay una capa de atención que ayuda al decodificador a centrarse en partes relevantes de la oración de entrada.

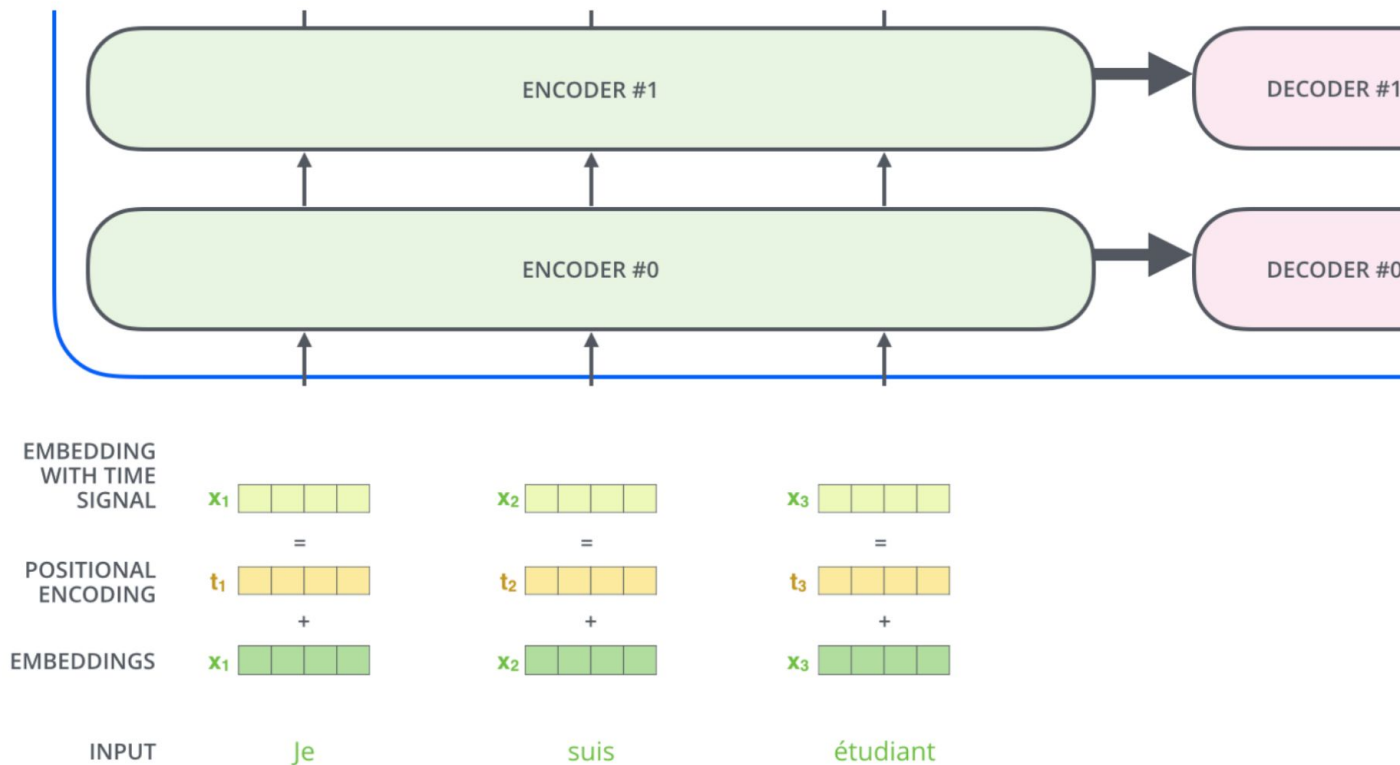


Vectores latentes

- Cada palabra en cada posición sigue su propio camino en el codificador.
- Hay dependencias entre estos caminos en la capa de autoatención.
- La capa FF no tiene estas dependencias, permitiendo que los caminos se ejecuten en paralelo.

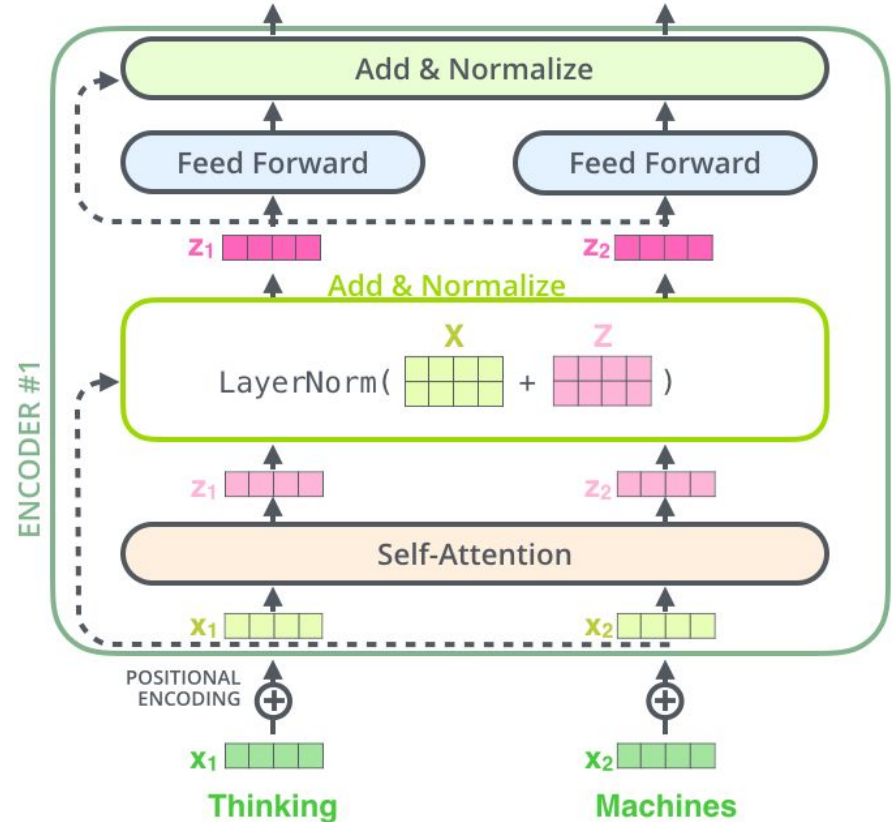


Embedding posicional

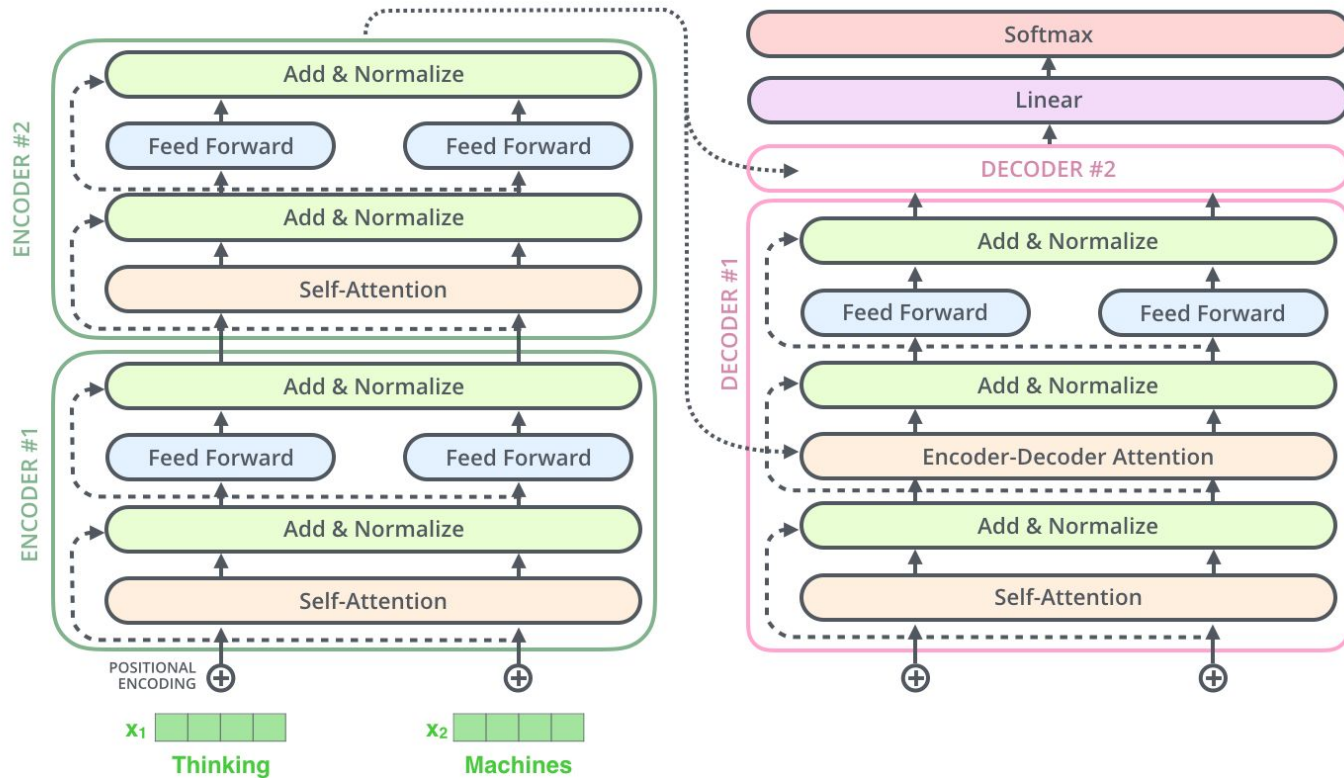


Residuales

- Cada subcapa (autoatención, ffn) en cada Encoder tiene conexiones residuales.
- Hay un paso de normalización de capa.
- Idem subcapas del decodificador.

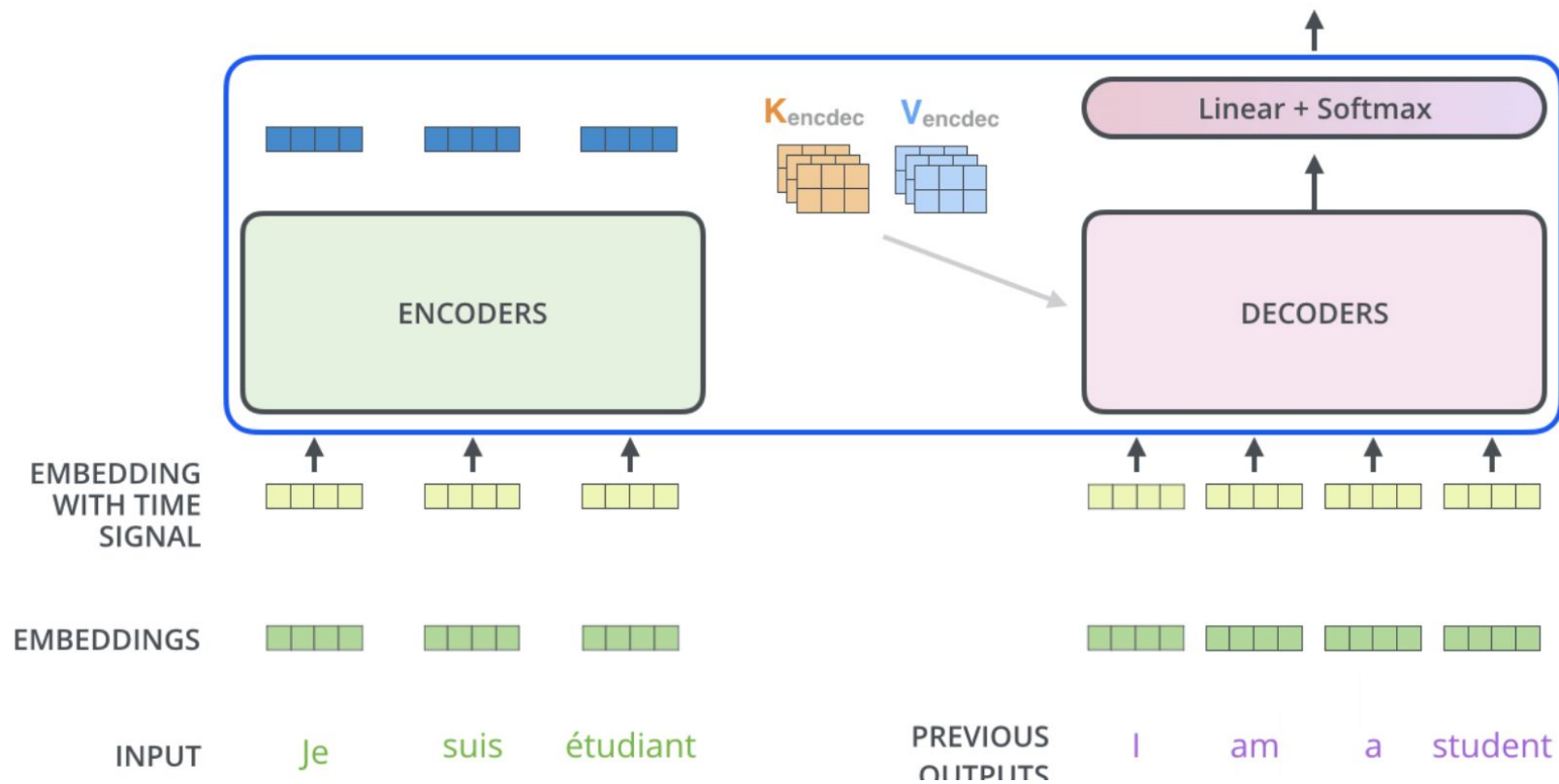


Ejemplo transformer 2 Enc, 2 Dec



Decoding time step: 1 2 3 4 **5** 6

OUTPUT I am a student <end of sentence>



Resultado

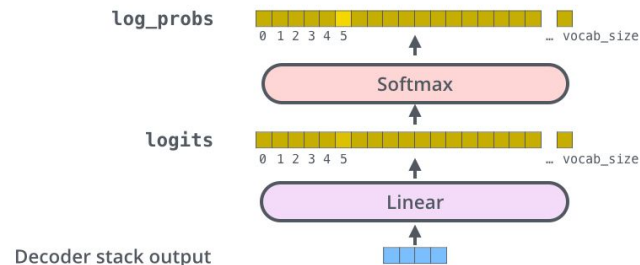
- El decodificador produce un vector de valores flotantes como salida.
 - a. La capa lineal FC, seguida por una capa softmax, se encarga de convertir ese vector en una palabra.
 - b. proyecta el vector producido en un vector mucho más grande llamado vector de logits.

Which word in our vocabulary is associated with this index?

am

Get the index of the cell with the highest value (argmax)

5



Resultado

- Supongamos que el modelo conoce 10,000 palabras únicas en inglés (su "vocabulario de salida").
 - a. El vector de logits tendría 10,000 celdas, cada una correspondiente a la puntuación de una palabra única.
 - b. La capa softmax convierte esas puntuaciones en probabilidades (todas positivas y suman 1.0).

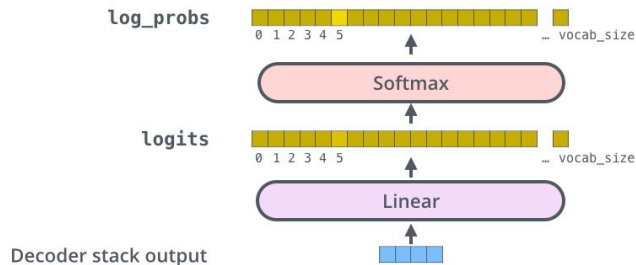
Se elige la celda con la probabilidad más alta, y la palabra asociada se produce como la salida para ese paso de tiempo.

Which word in our vocabulary is associated with this index?

am

Get the index of the cell with the highest value (argmax)

5



Resultado

Output Vocabulary

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word "am"

0.0	1.0	0.0	0.0	0.0	0.0
-----	-----	-----	-----	-----	-----

Agenda

1. Attention
2. Transformers
3. **BERT**
4. Vision Transformer (ViT)
5. Bibliografía

Bidirectional Encoder Representations from Transformers (BERT)

- **Bidireccionalidad:** BERT tiene la capacidad de procesar información en ambas direcciones dentro de una secuencia de palabras. Esto significa que puede entender el contexto de una palabra no solo por las palabras que la preceden, sino también por las que la siguen.
- **Atención Distribuida:** BERT utiliza una atención distribuida basada en el mecanismo de atención de los Transformers. Esta capacidad le permite asignar diferentes pesos a diferentes partes de la entrada, enfocándose en las partes más relevantes.

Bidirectional Encoder Representations from Transformers (BERT)

- **Pre-entrenamiento:** BERT se entrena previamente en grandes cantidades de datos no etiquetados antes de ser afinado para tareas específicas. Durante el pre-entrenamiento, el modelo aprende representaciones de palabras contextualizadas.
- **Transfer learning:** Después del pre-entrenamiento, BERT se puede refinar para tareas específicas, como clasificación de texto, etiquetado de entidades, respuesta a preguntas, entre otras. Este refinamiento implica entrenar el modelo en un conjunto de datos específico para la tarea deseada.

Entrada BERT

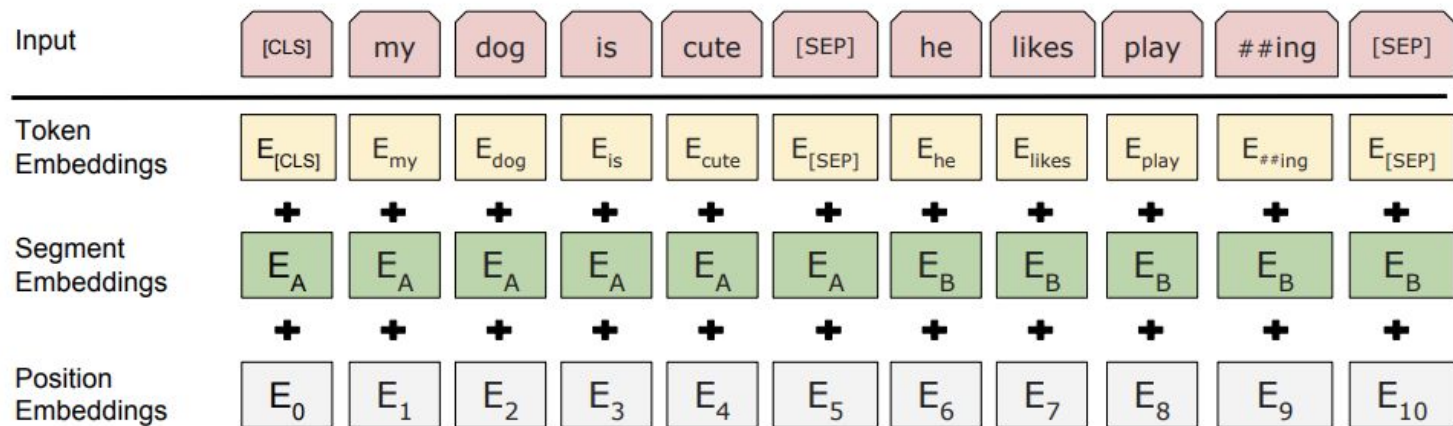


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

Entrenamiento

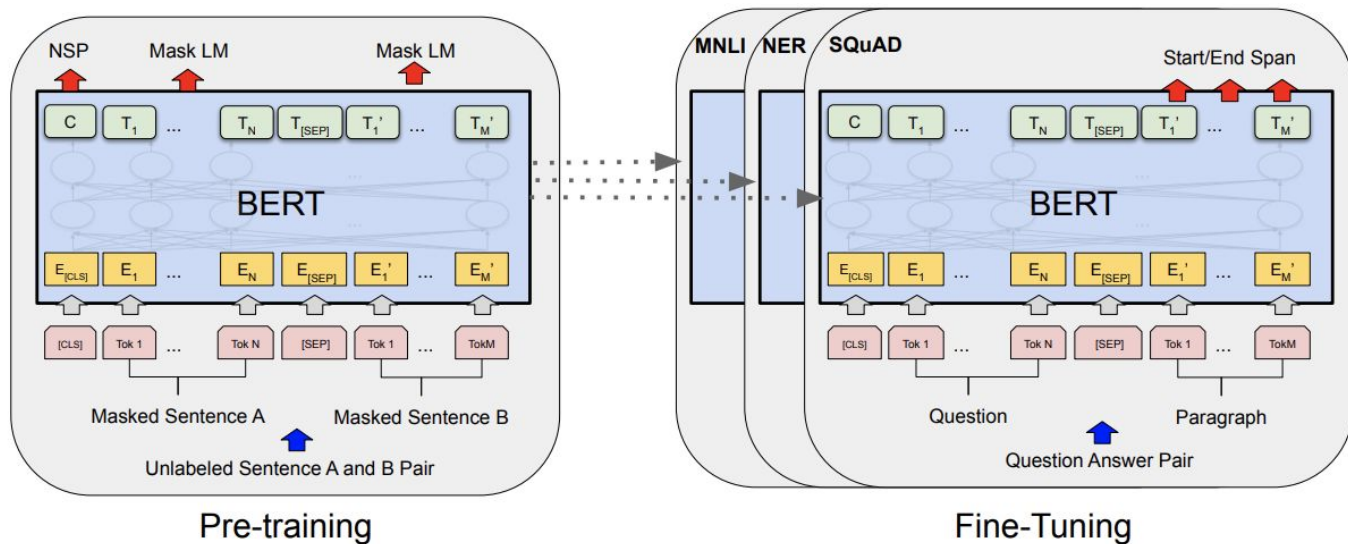


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

Pre-Training

Masked LM and the Masking Procedure Assuming the unlabeled sentence is my dog is hairy, and during the random masking procedure we chose the 4-th token (which corresponding to hairy), our masking procedure can be further illustrated by

- 80% of the time: Replace the word with the [MASK] token, e.g., my dog is hairy → my dog is [MASK]
- 10% of the time: Replace the word with a random word, e.g., my dog is hairy → my dog is apple
- 10% of the time: Keep the word unchanged, e.g., my dog is hairy → my dog is hairy. The purpose of this is to bias the representation towards the actual observed word.

Pre-Training

Next Sentence Prediction The next sentence prediction task can be illustrated in the following examples.

Input = [CLS] the man went to [MASK] store [SEP]
 he bought a gallon [MASK] milk [SEP]

Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]
 penguin [MASK] are flight ##less birds [SEP]

Label = NotNext

Fine-tuning

- Fine-tuning es un tipo de Transfer learning
- Partimos de una red ya entrenada para otra tarea, y la entrenamos ahora para una nueva (con otros datos, en general)
- Otros modos de transfer learning es mantener ciertas capas congeladas y solamente entrenar algunas (en general las últimas, que realizan las predicciones finales)

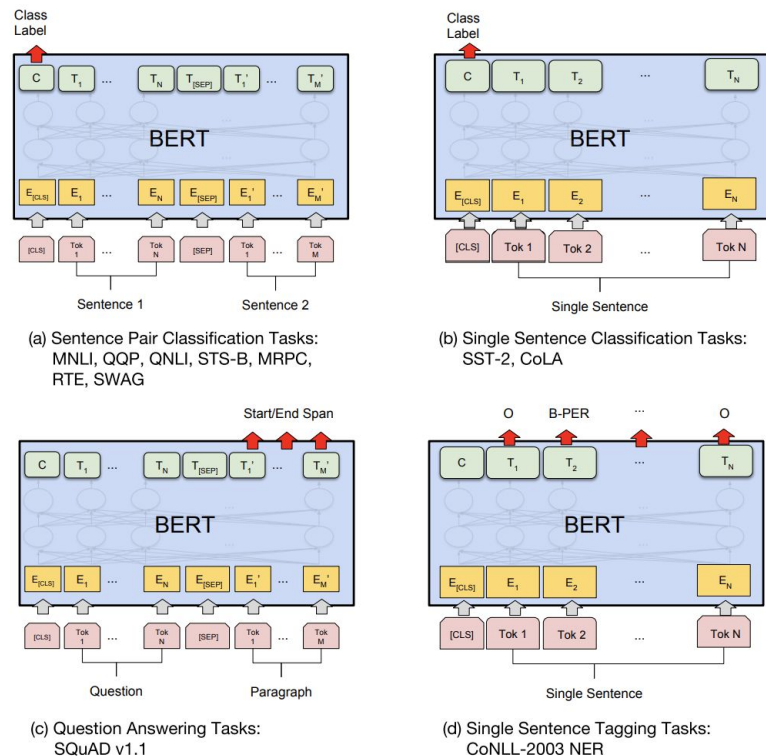
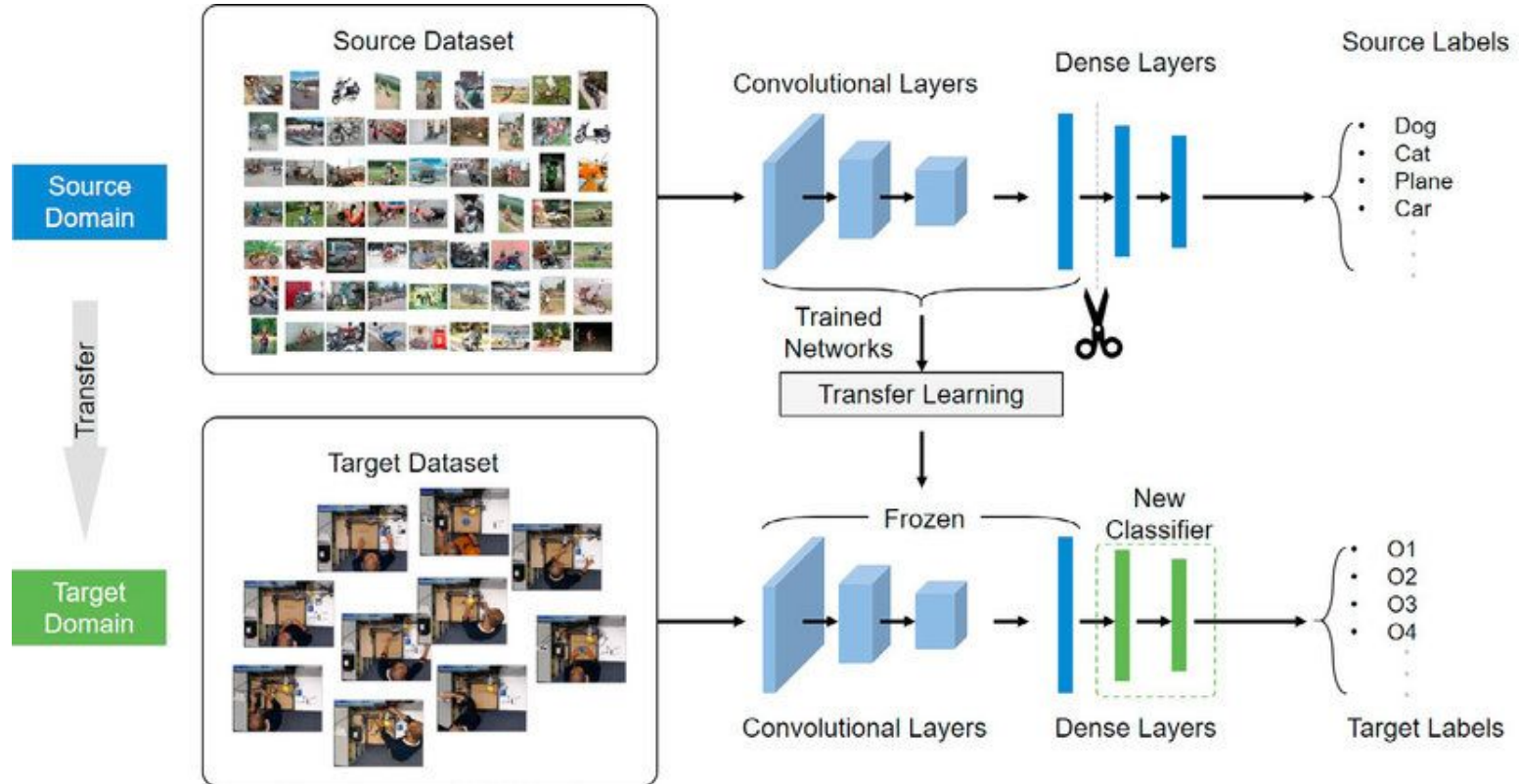


Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

(Transfer learning)



Parámetros

Model	Total train compute (PF-days)	Total train compute (flops)	Params (M)	Training tokens (billions)	Flops per param per token	Mult for bwd pass	Fwd-pass flops per active param per token	Frac of params active for each token
T5-Small	2.08E+00	1.80E+20	60	1,000	3	3	1	0.5
T5-Base	7.64E+00	6.60E+20	220	1,000	3	3	1	0.5
T5-Large	2.67E+01	2.31E+21	770	1,000	3	3	1	0.5
T5-3B	1.04E+02	9.00E+21	3,000	1,000	3	3	1	0.5
T5-11B	3.82E+02	3.30E+22	11,000	1,000	3	3	1	0.5
BERT-Base	1.89E+00	1.64E+20	109	250	6	3	2	1.0
BERT-Large	6.16E+00	5.33E+20	355	250	6	3	2	1.0
RoBERTa-Base	1.74E+01	1.50E+21	125	2,000	6	3	2	1.0
RoBERTa-Large	4.93E+01	4.26E+21	355	2,000	6	3	2	1.0
GPT-3 Small	2.60E+00	2.25E+20	125	300	6	3	2	1.0
GPT-3 Medium	7.42E+00	6.41E+20	356	300	6	3	2	1.0
GPT-3 Large	1.58E+01	1.37E+21	760	300	6	3	2	1.0
GPT-3 XL	2.75E+01	2.38E+21	1,320	300	6	3	2	1.0
GPT-3 2.7B	5.52E+01	4.77E+21	2,650	300	6	3	2	1.0
GPT-3 6.7B	1.39E+02	1.20E+22	6,660	300	6	3	2	1.0
GPT-3 13B	2.68E+02	2.31E+22	12,850	300	6	3	2	1.0
GPT-3 175B	3.64E+03	3.14E+23	174,600	300	6	3	2	1.0

Table D.1: Starting from the right hand side and moving left, we begin with the number of training tokens that each model was trained with. Next we note that since T5 uses an encoder-decoder model, only half of the parameters are active for each token during a forward or backwards pass. We then note that each token is involved in a single addition and a single multiply for each active parameter in the forward pass (ignoring attention). Then we add a multiplier of 3x to account for the backwards pass (as computing both $\frac{\partial \text{params}}{\partial \text{loss}}$ and $\frac{\partial \text{acts}}{\partial \text{loss}}$ use a similar amount of compute as the forwards pass. Combining the previous two numbers, we get the total flops per parameter per token. We multiply this value by the total training tokens and the total parameters to yield the number of total flops used during training. We report both flops and petaflop/s-day (each of which are $8.64\text{e}+19$ flops).

Language Models are Few-Shot Learners

Tom B. Brown*	Benjamin Mann*	Nick Ryder*	Melanie Subbiah*	
Jared Kaplan[†]	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Girish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess		Jack Clark	Christopher Berner	
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei	

OpenAI

Abstract

Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions – something which current NLP systems still largely struggle to do. Here we show that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically, our GPT-3 model generates language model with 175 billion

LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron*, Thibaut Lavril*, Gautier Izacard*, Xavier Martinet
 Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal
 Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin
 Edouard Grave*, Guillaume Lample*

Meta AI

Abstract

We introduce LLaMA, a collection of foundation language models ranging from 7B to 65B parameters. We train our models on trillions of tokens, and show that it is possible to train state-of-the-art models using publicly available datasets exclusively, without resorting to proprietary and inaccessible datasets. In particular, LLaMA-13B outperforms GPT-3 (175B) on most benchmarks, and LLaMA-65B is competitive with the best models, Chinchilla-70B and PaLM-540B. We release all our models to the research community¹.

1 Introduction

Large Languages Models (LLMs) trained on massive corpora of texts have shown their ability to perform new tasks from textual instructions or from a few examples (Brown et al., 2020). These few-shot properties first appeared when scaling models to a sufficient size (Kaplan et al., 2020), resulting in a line of work that focuses on further scaling these models (Chowdhery et al., 2022; Rae et al., 2021). These efforts are based on the assumption that

performance, a smaller one trained longer will ultimately be cheaper at inference. For instance, although Hoffmann et al. (2022) recommends training a 10B model on 200B tokens, we find that the performance of a 7B model continues to improve even after 1T tokens.

The focus of this work is to train a series of language models that achieve the best possible performance at various inference budgets, by training on more tokens than what is typically used. The resulting models, called *LLaMA*, ranges from 7B to 65B parameters with competitive performance compared to the best existing LLMs. For instance, LLaMA-13B outperforms GPT-3 on most benchmarks, despite being 10× smaller. We believe that this model will help democratize the access and study of LLMs, since it can be run on a single GPU. At the higher-end of the scale, our 65B-parameter model is also competitive with the best large language models such as Chinchilla or PaLM-540B.

Unlike Chinchilla, PaLM, or GPT-3, we only use publicly available data, making our work com-



Meta INTRODUCING



LLAMA 3.2



Introducing Llama 3.2

The open-source AI model you can fine-tune, distill and deploy anywhere is now available in more versions. Choose from 1B, 3B, 11B or 90B, or continue building with Llama 3.1

[Download models](#)[Try Llama on Meta AI](#)

See how Llama is the leading open source model family

[Learn more](#)

Latest models

Llama 3.2 includes multilingual text-only models (1B, 3B) and text-image models (11B, 90B), with quantized versions of 1B and 3B offering on average up to 56% smaller size and 2-3x speedup, ideal for on-device and edge deployments.

[Start building](#)

Lightweight

1B and 3B

Our lightweight and most efficient models you can run everywhere on mobile and on edge devices.

[Download models](#)

Multimodal

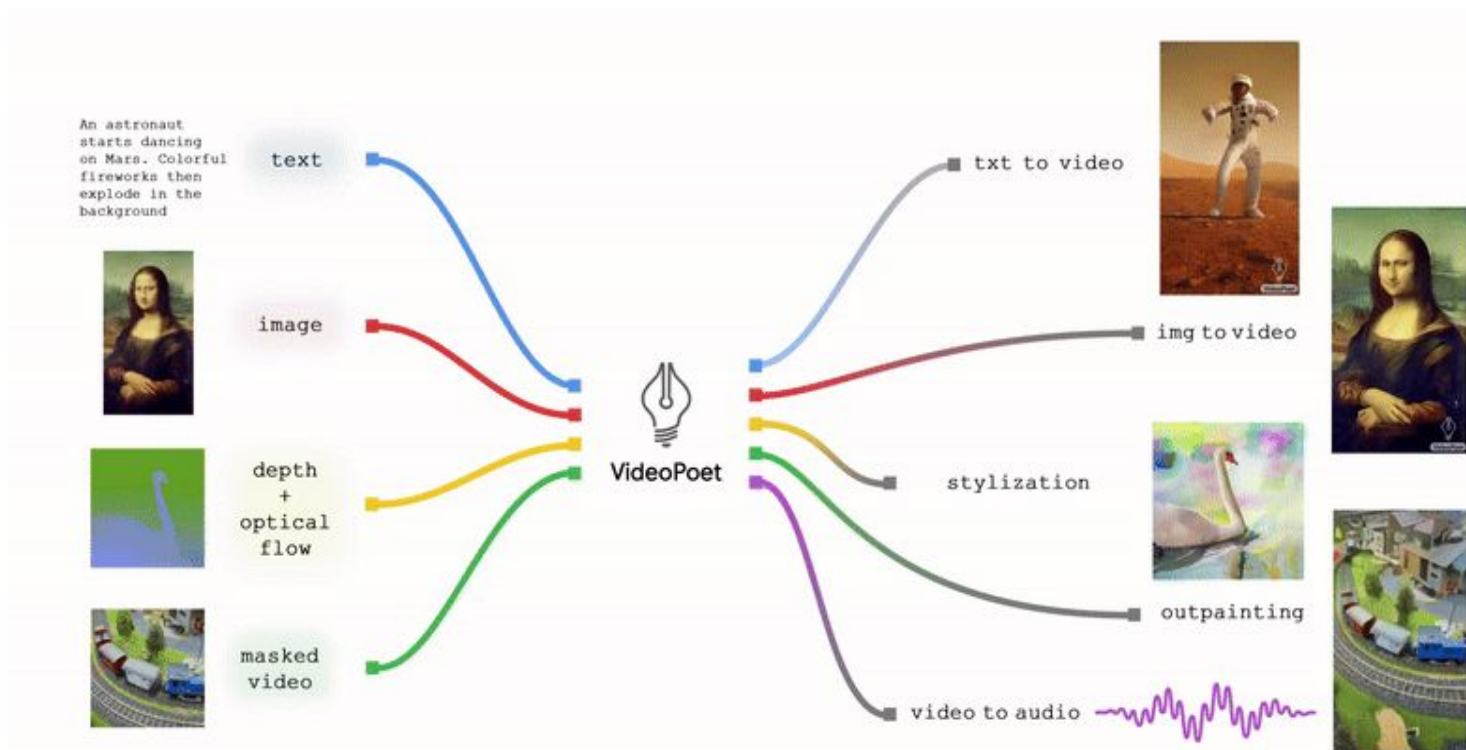
11B and 90B

Our open multimodal models that are flexible and can reason on high resolution images.

[Download models](#)

LLMs de texto, imágenes, audio, video, música, ...

<https://sites.research.google/videopoet/>



Agenda

1. Attention
2. Transformers
3. BERT
- 4. Vision Transformer (ViT)**
5. Bibliografía

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.¹

1 INTRODUCTION

Self-attention-based architectures, in particular Transformers (Vaswani et al., 2017), have become the model of choice in natural language processing (NLP). The dominant approach is to pre-train on a large text corpus and then fine-tune on a smaller task-specific dataset (Devlin et al., 2019). Thanks to Transformers’ computational efficiency and scalability, it has become possible to train models of unprecedented size, with over 100B parameters (Brown et al., 2020; Lepikhin et al., 2020). With the models and datasets growing, there is still no sign of saturating performance.

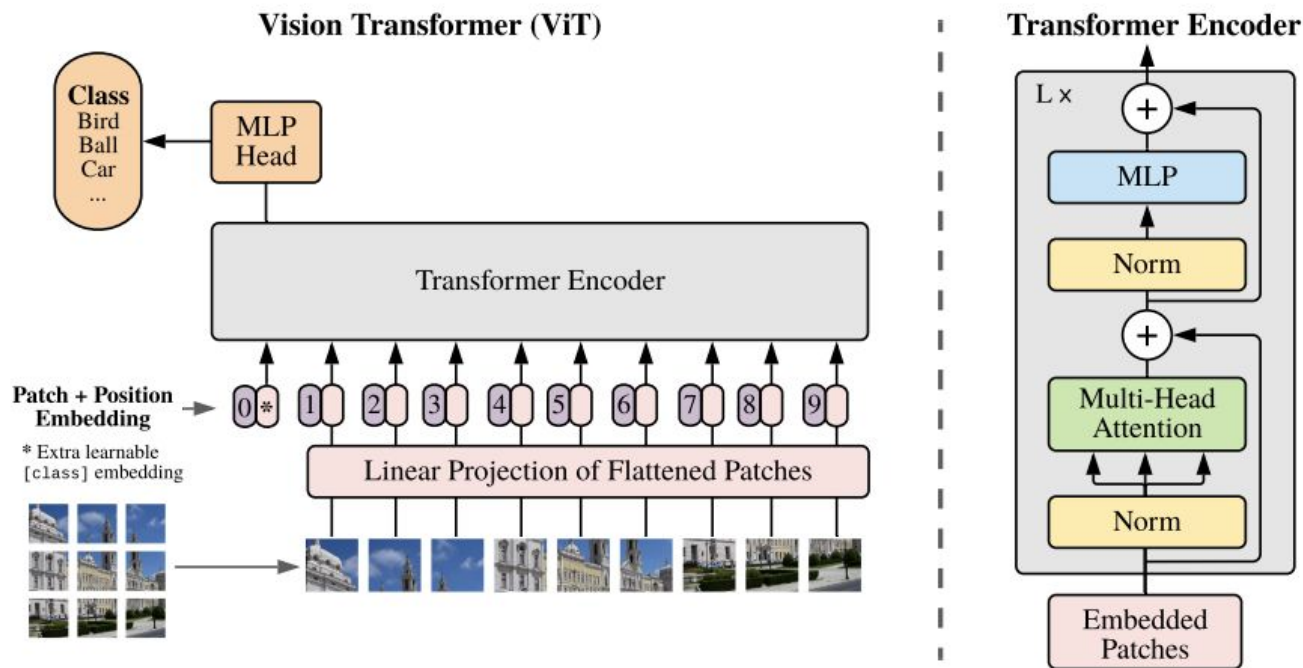


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

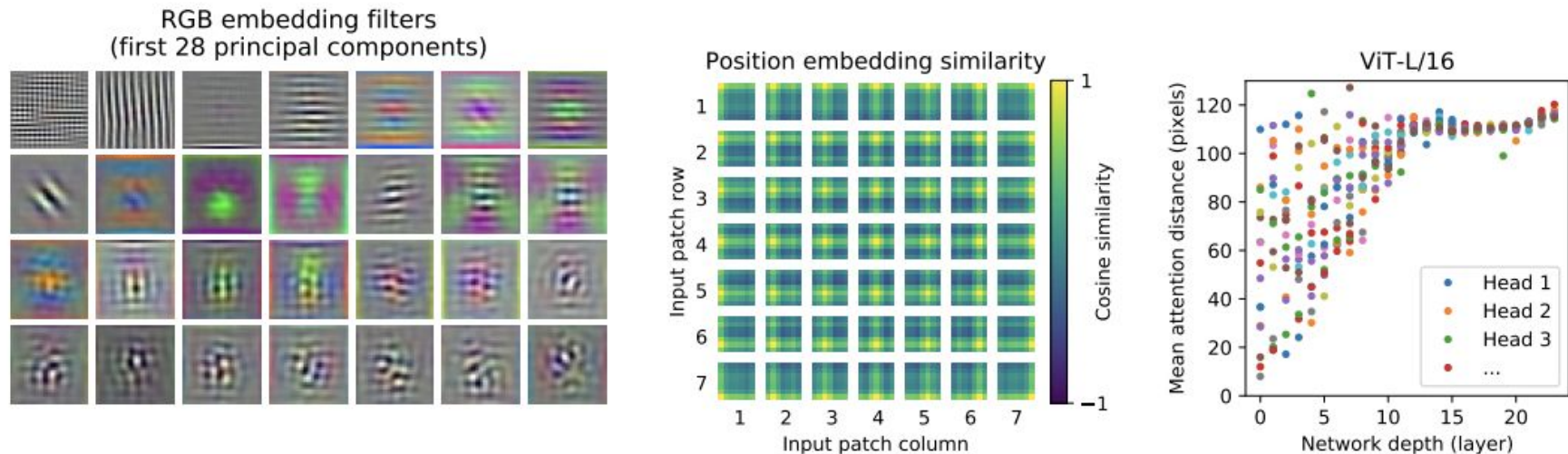


Figure 7: **Left:** Filters of the initial linear embedding of RGB values of ViT-L/32. **Center:** Similarity of position embeddings of ViT-L/32. Tiles show the cosine similarity between the position embedding of the patch with the indicated row and column and the position embeddings of all other patches. **Right:** Size of attended area by head and network depth. Each dot shows the mean attention distance across images for one of 16 heads at one layer. See Appendix D.7 for details.

Bibliografía

Transformer:

<https://arxiv.org/pdf/1706.03762.pdf>

<https://jalammar.github.io/illustrated-transformer/>

Bert: <https://arxiv.org/abs/1810.04805>

GPT3 : <https://arxiv.org/pdf/2005.14165.pdf>

LLAMA: <https://arxiv.org/pdf/2302.13971.pdf>

VIT: <https://arxiv.org/pdf/2010.11929.pdf>